Machine Learning

Supervised Learning: Logistic Regression

Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

Outcome

Session Outcome

- Understand logistic regression for classification and explain how decision boundaries are learned via probability-based predictions.

Logistic Regression

Logistic Regression

Can we use regression for classification applications?

Can we use a Straight Line to 0/1 Labels?

Take the placement data. The target is now a label: $y = 1$ (placed), $y = 0$ (not placed).

If we fit the straight line $\hat{y} = \theta_0 + \theta_1 x$ to these 0/1 values:

- The output is *not bounded*. For a low CGPA the line gives $\hat{y} < 0$; for a high CGPA it gives $\hat{y} > 1$.
  - A negative probability or a probability of 1.2 has *no meaning*.
- The line has no natural *cut-off*. Where exactly does "placed" begin?

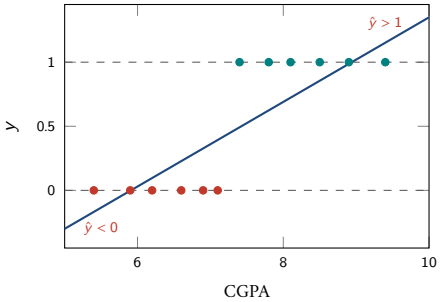


Figure: A straight line fitted to 0/1 labels leaves the range $[0, 1]$

Question: How do we force the output of $\boldsymbol{\theta}^\top \boldsymbol{x}$ to stay between 0 and 1, so that it can be read as a probability?

Logistic Regression

Logistic Regression is a supervised machine learning algorithm used to predict the *probability* that an input belongs to a class. The linear score $\boldsymbol{\theta}^\top \boldsymbol{x}$ is first computed as before, and is then squashed into the range $(0, 1)$ by the *sigmoid* function.

The logistic regression model prediction is given by:

$$\hat{p} \;=\; h_\theta(\boldsymbol{x}) \;=\; \sigma\big(\boldsymbol{\theta}^\top \boldsymbol{x}\big) \;=\; \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \boldsymbol{x}}}$$

where

- $\hat{p} = P(y = 1 \mid \boldsymbol{x})$ is the predicted probability of class 1
- $\boldsymbol{\theta} = [\theta_0, \theta_1, \ldots, \theta_d]^T$ is the parameter vector
- $\boldsymbol{x} = [1, x_1, \ldots, x_d]^T$ is the feature vector
- $z = \boldsymbol{\theta}^\top \boldsymbol{x}$ is called the *score* or *logit*

Note: in spite of its name, logistic regression is used for **classification**, not for predicting numbers.

The Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output always lies in $(0, 1)$, for every real z
- $\sigma(0) = 0.5$
- $\sigma(z) \to 1$ as $z \to +\infty$, $\sigma(z) \to 0$ as $z \to -\infty$
- $\sigma(-z) = 1 - \sigma(z)$
- Derivative: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$
- It is *monotonic*: a larger score always gives a larger probability

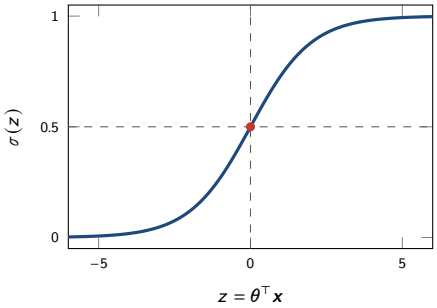


Figure: The sigmoid squashes any score into $(0, 1)$

Decision Rule

The model returns a probability. To take a decision we compare it with a **threshold** τ (usually 0.5):

$$\hat{y} = \begin{cases} 1 \text{ (placed)}, & \text{if } \hat{p} \geq 0.5 \\ 0 \text{ (not placed)}, & \text{if } \hat{p} < 0.5 \end{cases}$$

Since σ is monotonic and $\sigma(0) = 0.5$,

$$\hat{p} \geq 0.5 \iff z = \boldsymbol{\theta}^\top \boldsymbol{x} \geq 0$$

- So the decision can be taken directly from the *sign of the score*.
- The threshold need not be 0.5. If a wrong "not placed" is costly, lower it.
- The probability itself is more useful than the label: it tells the placement cell *how confident* the model is.

Log-Odds

Invert the sigmoid. From $\hat{p} = \sigma(z)$,

$$\frac{\hat{p}}{1-\hat{p}} = e^{z} = e^{\theta^{\top}\mathbf{x}} \quad \Longrightarrow \quad \underbrace{\ln \frac{\hat{p}}{1-\hat{p}}}_{\text{log-odds (logit)}} = \theta^{\top}\mathbf{x}$$

- Logistic regression is a *linear model of the log-odds*.
- θ_j = change in log-odds per unit of feature j.

z	odds	$\hat{p}$
−2	0.14	0.12
−0.12	0.89	0.47
0	1	0.50
1	2.72	0.73
4.70	110	0.99

Why not use MSE as the Cost?

For linear regression we minimised $J = \frac{1}{m}\sum(\boldsymbol{\theta}^\top \boldsymbol{x}^{(i)} - y^{(i)})^2$. Can we do the same here, with $\sigma(\boldsymbol{\theta}^\top \boldsymbol{x})$ in place of $\boldsymbol{\theta}^\top \boldsymbol{x}$?

- The sigmoid is a *nonlinear* function of $\boldsymbol{\theta}$. Squaring its error gives a cost surface that is *not convex*: it has several local minima.
- Gradient descent may then stop at a local minimum and never reach the best parameters.
- Also, MSE punishes a badly wrong *confident* prediction only mildly, whereas in classification such a mistake is the most serious one.

Question: which cost function is convex for logistic regression, and punishes confident mistakes heavily?

Answer: the **cross-entropy** (also called *log loss*).

Cross-Entropy (Log Loss)

Cost for a single training example:

$$\text{cost}(\hat{p}, y) = \begin{cases} -\ln(\hat{p}), & \text{if } y = 1 \\ -\ln(1-\hat{p}), & \text{if } y = 0 \end{cases}$$

Both cases can be written in one line, because one factor always switches off:

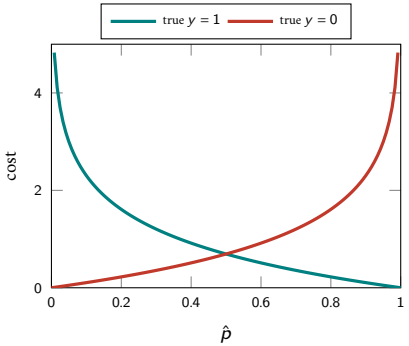
$$\text{cost} = -\left[\, y \ln \hat{p} + (1-y) \ln(1-\hat{p})\right]$$

Averaging over all m training examples gives the cost function

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \ln \hat{p}^{(i)} + (1-y^{(i)}) \ln\left(1-\hat{p}^{(i)}\right) \right]$$

where $\hat{p}^{(i)} = \sigma\left(\theta^{\top} \boldsymbol{x}^{(i)}\right)$ is the probability predicted for the i^{th} example.

Cross-Entropy



- Correct *and* confident ($y = 1$, $\hat{p} \to 1$): the cost falls to 0.
- Wrong *and* confident ($y = 1$, $\hat{p} \to 0$): the cost grows without limit.
- Sitting on the fence ($\hat{p} = 0.5$) always costs $\ln 2 = 0.6931$.
- So the model is pushed to be confident *only* when it is right.
- $J(\theta)$ is **convex**: there is a single global minimum, exactly as in linear regression.

Gradient Descent for Logistic Regression

Differentiating J and using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the sigmoid derivative cancels and we are left with a very simple result:

$$\nabla_\theta J = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}^{(i)} - y^{(i)} \right) \boldsymbol{x}^{(i)} = \frac{1}{m} \boldsymbol{X}^\top (\sigma(\boldsymbol{X}\theta) - \boldsymbol{y})$$

The update rule is the same as in linear regression:

$$\theta^{(i+1)} = \theta^{(i)} - \eta \, \nabla_\theta J\left(\theta^{(i)}\right)$$

- Only $\boldsymbol{X}\theta$ has been replaced by $\sigma(\boldsymbol{X}\theta)$; everything else is unchanged.
- $(\hat{p} - y)$ is the *error*: the amount by which the predicted probability misses the true label.
- **There is no Normal Equation here.** σ is nonlinear in θ, so setting $\nabla_\theta J = 0$ cannot be solved in closed form. Logistic regression *must* be trained iteratively.

Example

Dataset: Pass or fail in an exam for 5 students.

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

Assume the model suggested by the optimizer for odds of passing the course is:

$$\log(odds) = z = -64 + 2 * hours$$

Part 1

Question 1: Calculate the probability of pass for the student who studied 33 hours.

Part 1

Question 1: Calculate the probability of pass for the student who studied 33 hours. **Step 1: Calculate** z

$$z = -64 + 2 * 33$$

$$z = -64 + 66 = 2$$

Step 2: Apply Sigmoid Function

$$p = \frac{1}{1 + e^{-z}}$$

$$p = \frac{1}{1 + e^{-2}} = 0.88$$

Conclusion: If a student studies 33 hours, there is an **88% chance** that the student will pass the exam.

Part 2

Question 2: At least how many hours should a student study to pass the course with a probability of more than 95%?

Part 2

Question 2: At least how many hours should a student study to pass the course with a probability of more than 95%?

Set up the Hypothesis

$$h_\theta(\boldsymbol{x}) = p = \frac{1}{1 + e^{-z}} = 0.95$$

Solve for z

$$0.95 * (1 + e^{-z}) = 1$$

$$0.95 * e^{-z} = 1 - 0.95 = 0.05$$

$$e^{-z} = \frac{0.05}{0.95} = 0.0526$$

$$\ln(e^{-z}) = \ln(0.0526)$$

$$-z = -2.94 \implies z = 2.94$$

Part 2

Solve for hours

$$\log(odds) = z = -64 + 2 * hours$$

Substitute $z = 2.94$:

$$2.94 = -64 + 2 * hours$$

$$2 * hours = 2.94 + 64$$

$$2 * hours = 66.94$$

$$hours = \frac{66.94}{2}$$

Conclusion:

$$hours = 33.47 \text{ Hours}$$

Linear vs Logistic Regression

	Linear regression	**Logistic regression**
Target	$y \in \mathbb{R}$ (salary)	$y \in \{0, 1\}$ (placed)
Model	$\hat{y} = \theta^\top \mathbf{x}$	$\hat{p} = \sigma(\theta^\top \mathbf{x})$
Linear in	the prediction	the log-odds
Loss	MSE	cross-entropy
Convex?	yes	yes
Closed form	yes, normal equations	**no** — iterative only
Gradient	$\frac{2}{n}\mathbf{X}^\top(\mathbf{X}\theta - \mathbf{y})$	$\frac{1}{n}\mathbf{X}^\top(\sigma(\mathbf{X}\theta) - \mathbf{y})$
Output used for	a number	a probability + a threshold

Key Insight

Despite the name, logistic regression is a **classifier**. The word "regression" is still used because it regresses the log-odds on the features.

Appendix

Cross Entropy from Maximum Likelihood

Hypothesis Representation: Instead of a linear output $z = \theta^T \boldsymbol{x}$, we pass z through the *Sigmoid Function*:

$$\hat{p} = h_\theta(\boldsymbol{x}) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\theta^T \boldsymbol{x}}}$$

where:

- $\hat{p}$ is the predicted probability that $y = 1$ given $\boldsymbol{x}$.
- θ represents the model weights (parameters).
- $\boldsymbol{x}$ represents the input features.

The Probability Distribution

Since the outcome y is binary ($y \in \{0, 1\}$), it follows a Bernoulli distribution. We can write the probabilities as:

$$P(y = 1|\boldsymbol{x}; \boldsymbol{\theta}) = \hat{p}$$
$$P(y = 0|\boldsymbol{x}; \boldsymbol{\theta}) = 1 - \hat{p}$$

We can combine these into a single equation for the probability of a single training example $(y, \boldsymbol{x})$:

$$P(y|\boldsymbol{x}; \boldsymbol{\theta}) = (\hat{p})^y (1 - \hat{p})^{1-y}$$

Note: If $y = 1$, the second term becomes 1. If $y = 0$, the first term becomes 1.

The Likelihood Function

Maximum Likelihood Estimation (MLE) seeks to find the parameters θ that maximize the probability of observing the given training data.

Assuming we have m independent training examples, the **likelihood Function** $L(\theta)$ is the product of individual probabilities:

$$L(\theta) = \prod_{i=1}^{m} P(y^{(i)}|\boldsymbol{x}^{(i)}; \theta)$$

Substitute the Bernoulli probability:

$$L(\theta) = \prod_{i=1}^{m} \left(\hat{p}^{(i)}\right)^{y^{(i)}} \left(1 - \hat{p}^{(i)}\right)^{1-y^{(i)}}$$

The Log-Likelihood Function

To simplify the math, we take the natural logarithm to get the Log-Likelihood $\ell(\theta)$:

$$\ell(\theta) = \log L(\theta) = \log \left(\prod_{i=1}^{m} \left(\hat{p}^{(i)} \right)^{y^{(i)}} \left(1 - \hat{p}^{(i)} \right)^{1-y^{(i)}} \right)$$

Using logarithm rules ($\log(ab) = \log a + \log b$ and $\log(a^b) = b \log a$):

$$\ell(\theta) = \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

Our goal in MLE is to **maximize** $\ell(\theta)$.

Cross-Entropy Loss (Log Loss)

In machine learning, we typically formulate optimization as a *minimization* problem. Therefore, we minimize the negative log-likelihood.

We define the average Cost Function $J(\theta)$ over m examples as:

$$J(\theta) = -\frac{1}{m}\ell(\theta)$$

This gives us the familiar **Cross-Entropy Loss** (or Log Loss):

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{p}^{(i)}) + (1-y^{(i)})\log(1-\hat{p}^{(i)})\right]$$

Derivative of the Sigmoid Function

Before calculating the gradient of $J(\theta)$, we need the derivative of the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$.

$$
\begin{aligned}
\frac{d}{dz}\sigma(z) &= \frac{d}{dz}(1+e^{-z})^{-1} \\
&= -(1+e^{-z})^{-2}(-e^{-z}) \\
&= \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right) \\
&= \sigma(z)(1-\sigma(z))
\end{aligned}
$$

Gradient of the Cost Function

We want to find the partial derivative of $J(\boldsymbol{\theta})$ with respect to a specific weight θ_j. By the chain rule:

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial J}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z} \cdot \frac{\partial z}{\partial \theta_j}$$

Let's compute each part for a single training example (x, y):

- $\frac{\partial J}{\partial \hat{p}} = -\left(\frac{y}{\hat{p}} - \frac{1-y}{1-\hat{p}}\right) = \frac{\hat{p}-y}{\hat{p}(1-\hat{p})}$
- $\frac{\partial \hat{p}}{\partial z} = \hat{p}(1-\hat{p})$ $\quad (\hat{p} = \sigma(z)$ from previous slide)
- $\frac{\partial z}{\partial \theta_j} = \frac{\partial}{\partial \theta_j}(\boldsymbol{\theta}^T \boldsymbol{x}) = x_j$

Gradient of the Cost Function

Multiplying the terms together for a single example:

$$\begin{aligned}\frac{\partial J}{\partial \theta_j} &= \left(\frac{\hat{p} - y}{\hat{p}(1-\hat{p})}\right) \cdot (\hat{p}(1-\hat{p})) \cdot x_j \\ &= (\hat{p} - y)x_j\end{aligned}$$

Averaging this over all m training examples, the final gradient is:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}^{(i)} - y^{(i)}\right) x_j^{(i)}$$

Gradient Descent Update

Now that we have the gradient derived from the Maximum Likelihood Estimation, we can use Gradient Descent to iteratively update the parameters and minimize the Cross-Entropy Loss.

Update Rule:

$$\theta_j := \theta_j - \eta \frac{\partial J}{\partial \theta_j}$$

Substituting the derived gradient:

$$\theta_j = \theta_j - \eta \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

Where η is the learning rate.